

Kubernetes

1. Kubernetes Nedir?

Kubernetes, hem bildirimsel (declarative) yapılandırmayı hem de otomasyonu kolaylaştıran, konteynerleştirilmiş iş yüklerini ve hizmetlerini yönetmek için taşınabilir, genişletilebilir, açık kaynaklı bir platformdur. Geniş ve hızla büyüyen bir ekosisteme sahiptir. Kubernetes hizmetleri, desteği ve araçları yaygın olarak mevcuttur.

Kubernetes adı, Yunanca da dümenci veya pilot anlamına gelir. K8S kısaltması, “K” ve “s” harflerinin arasındaki sekiz harfin sayılmasıyla elde edilir.

Kubernetes, Google tarafından 6 Haziran 2014’te duyuruldu. Proje, Google çalışanları Joe Beda, Brendan Burns ve Craig McLuckie tarafından tasarlandı ve oluşturuldu. 1.0 versiyonunu 21 Temmuz 2015’te yayınladı. Google, Cloud Native Computing Foundation (CNCF) oluşturmak için Linux Foundation ile çalıştı ve Kubernetes’i başlangıç teknolojisi olarak sundu.

2. Neden Kubernetes’e İhtiyacınız Var?

Konteynerler, uygulamalarınızı paketlemek ve çalıştırmak için iyi bir yoldur. Üretim ortamında, uygulamaları çalıştıran konteynerleri yönetmeniz ve kesinti olmamasını sağlamanız gerekir. Örneğin, bir konteyner çökerse, başka bir konteynerin başlatılması gerekir. Bu davranışın bir sistem tarafından yönetilmesi daha kolay olmaz mıydı?

İşte Kubernetes burada devreye giriyor! Kubernetes, dağıtılmış sistemleri dayanıklı bir şekilde çalıştırmak için bir çerçeve sunar. Uygulamanız için ölçeklendirme ve arıza durumunda yedekleme işlemlerini üstlenir, dağıtım modelleri sağlar ve daha fazlasını sunar.

Kubernetes size şunları sağlar:

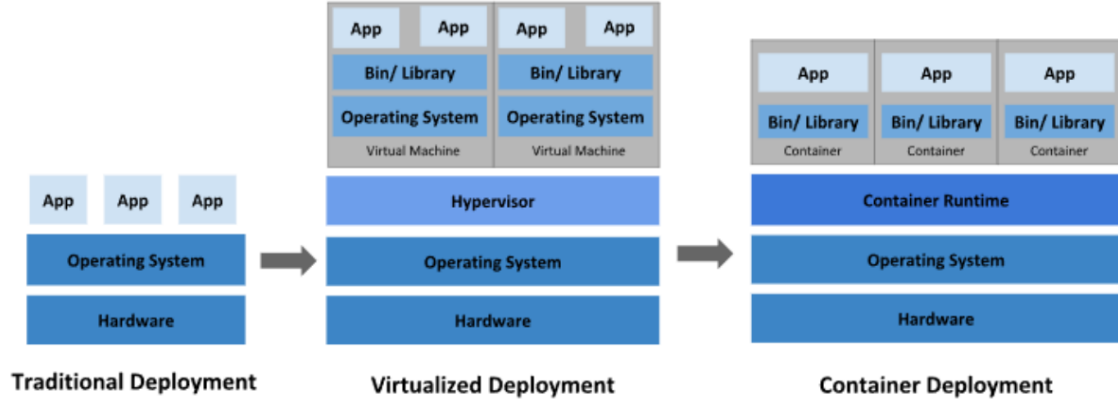
- **Service Discovery and Load Balancing (Servis Keşfi ve Yük Dengeleme):**
Kubernetes, bir konteyneri DNS adı veya IP adresi kullanarak kullanıma sunabilir.

Bir konteynere gelen trafik yüksekse, Kubernetes ağ trafiğini yük dengeleyerek ve dağıtarak dağıtımın istikrarlı olmasını sağlar.

- **Storage Orchestration (Depolama Orkestrasyonu):** Kubernetes, yerel depolama, genel bulut sağlayıcıları ve daha fazlası gibi seçtiğiniz bir depolama sistemini otomatik olarak bağlamanıza olanak tanır.
- **Automated Rollouts and Rollbacks (Otomatik Dağıtım ve Geri Alma):** Kubernetes kullanarak dağıtılan konteynerleriniz için istenen durumu tanımlayabilir ve gerçek durumu kontrollü bir hızda istenen duruma değiştirebilirsiniz. Örneğin, Kubernetes dağıtımınız içinde yeni konteynerler oluşturmak, mevcut konteynerleri kaldırmak ve tüm kaynaklarını yeni konteynere aktarmak için otomatikleştirebilirsiniz.
- **Automatic bin packing:** Kubernetes'e konteynerleştirilmiş görevleri çalıştırmak için kullanabileceği bir düğüm kümesi (node cluster, worker cluster, worker node) sağlarsınız. Kubernetes'e her konteynerin ne kadar CPU ve belleğe ihtiyacı olduğunu söylersiniz. Kubernetes, kaynaklarınızdan en iyi şekilde yararlanmak için konteynerleri düğümlerinize yerleştirebilir.
- **Self-Healing (Kendi kendini onarma):** Kubernetes, başarısız olan konteynerleri yeniden başlatır, konteynerleri değiştirir, kullanıcı tanımlı sağlık kontrolünüze yanıt vermeyen konteynerleri öldürür ve hizmet vermeye hazır olana kadar istemcilere duyurmaz.
- **Secret and Configuration Management (Gizlilik ve Yapılandırma Yönetimi):** Kubernetes parolalar, OAuth token'ları ve SSH anahtarları gibi hassas bilgileri saklamanıza ve yönetmenize olanak tanır. Konteyner görüntülerinizi yeniden oluşturmadan ve yapılandırmalarınızda gizlilikleri açığa çıkarmadan uygulama yapılandırmasını dağıtabilir ve güncelleyebilirsiniz.
- **Batch Execution (Toplu İşleme):** Servislere ek olarak, Kubernetes, istenirse başarısız olan konteynerleri değiştirerek toplu işlem ve CI iş yüklerinizi yönetebilir.
- **Horizontal Scaling or Vertical Scaling(Yatay Ölçeklendirme veya Dikey Ölçeklendirme):** Uygulamanızı basit bir komutla, kullanıcı arayüzüyle veya CPU kullanımına bağlı olarak otomatik olarak yukarı ve aşağı ölçeklendirin
- **IPv4/IPv6:** Pod'lara ve servislere IPv4 ve IPv6 adreslerinin tahsis edilmesi.

- **Designed for Extensibility (Geniřletilebilirlik için Tasarlanmıřtır):** Yukarı akıř kaynak kodunu deęiřtirmeden Kubernetes kümenize özellikler ekleyin.

3. Kubernetes'in Tarihsel Geliřim Süreci



řekil 2.1.

3.1. Geleneksel Daęıtım Dönemi (Traditional Deployment)

Kuruluřlar, bařlangıçta uygulamalarını fiziksel sunucular çalıřtırıyordu. Fiziksel bir sunucudaki uygulamalar için kaynak sınırlarını tanımlamanın bir yolu yoktur ve bu da kaynak tahsisi sorunlarına yol açıyordu. Örneęin fiziksel bir sunucuda birden fazla uygulama çalıřıyorsa, bir uygulamanın kaynakların çoęunun tükettięi ve bunun sonucunda dięer uygulamaların düşük performans gösterdięi durumlar olabilir. Bu soruna çözüm olarak geleneksel daęıtım döneminde, her uygulamayı farklı bir fiziksel sunucuda çalıřtırmaktı. Ancak her bir fiziksel sunucuya kurulan uygulamalar sunucu içerisindeki kaynakların tamamını kullanmadıkları için sunucularda atıl kapasite durumlarının ortaya çıkması ve kuruluřların her bir uygulama için kurmuř oldukları fiziksel sunucunun yönetimi maliyetli bir durum olduęu için bu yöntem ölçeklenemedi.

3.2. Sanallařtırılmıř Daęıtım Dönemi (Virtualized Deployment)

Çözüm olarak sanallařtırma tanıtıldı. Sanallařtırma, tek bir fiziksel sunucunun CPU'sunda birden fazla sanal makine çalıřtırmamıza olanak tanır. Sanallařtırma, uygulamaların sanal ortamlarda izole edilmesi olanak tanır ve bir uygulamanın bilgilerine bařka bir uygulama serbestçe eriřemedięinden belirli bir düzeyde güvenlik saęlar.

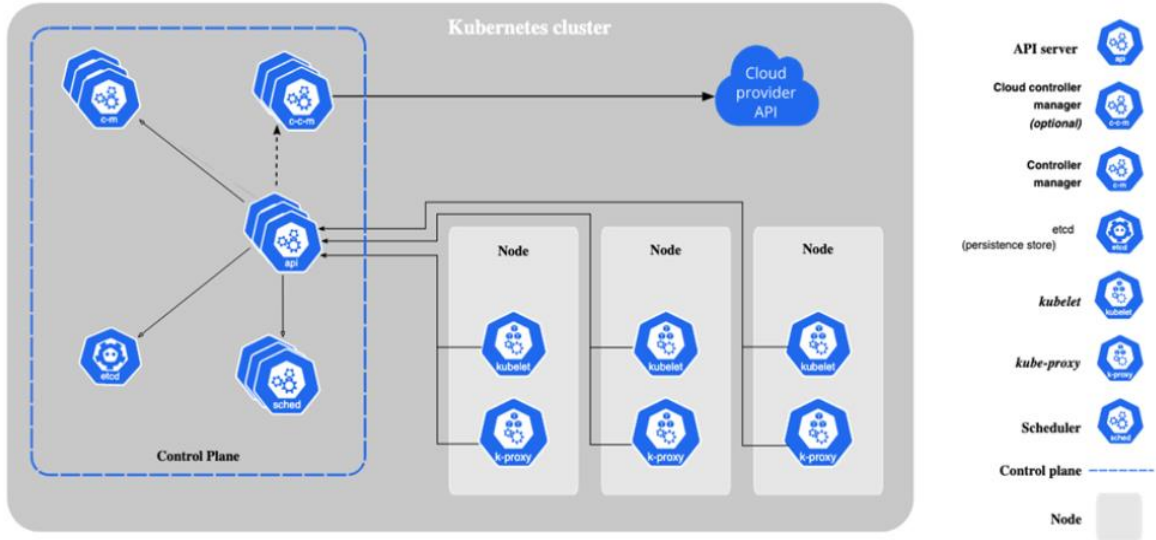
Sanallaştırma, fiziksel bir sunucudaki kaynakların daha iyi kullanılmasını sağladığı için donanım maliyetlerinin düşürülmesinde çok önemli bir rol oynar.

3.3. Konteyner Dağıtım Dönemi (Container Deployment)

Konteynerler sanal makinelere benzer, ancak işletim sistemini uygulamalar arasında paylaşmak için esnek yalıtım özelliklerine sahiptirler. Bu nedenle konteynerler hafif olarak kabul edilir. Konteynerler ekstra faydalar sağladıkları için daha popüler hale geldiler:

- **Agile Application Creation and Deployment (Çevik Uygulama Oluşturma ve Dağıtım):** Sanal makine imajı kullanımına kıyasla konteyner imajı oluşturmanın kolaylığı ve verimliliği artar.
- **Continuous Development, Integration and Deployment (Sürekli Geliştirme, Entegrasyon ve Dağıtım):** Güvenilir ve sık konteyner imajı oluşturma ve dağıtımı sağlar ve hızlı ve verimli geri alma işlemleri sunar.
- **Dev and Ops Separation of Concerns (Geliştirme ve Operasyon Sorumluluklarının Ayrılması):** Uygulama konteyner imajlarını dağıtım zamanında değil, derleme zamanında oluşturarak uygulamaları atyapıdan ayırır.
- **Observability (Gözlemlenebilirlik):** Yalnızca işletim sistemi düzeyindeki bilgileri ve metrikleri değil, aynı zamanda uygulama sağlığını ve diğer sinyalleri de ortaya çıkarır.
- **Environmental Consistency Across Development, Testing and Production (Geliştirme, Test ve Üretimde Ortam Tutarlılığı):** Dizüstü bilgisayarlarda olduğu gibi bulutta da aynı şekilde çalışır.
- **Cloud and OS Distribution Portability (Bulut ve İşletim Sistemi Dağıtım Taşınabilirliği):** Ubuntu, RHEL, CoreOS, şirket içi, büyük genel bulutlarda ve başka herhangi bir yerde çalışır.
- **Application-centric Management (Uygulama Merkezli Yönetim):** Bir işletim sistemini sanal donanımda çalıştırmaktan, bir uygulamayı mantıksal kaynaklar kullanarak bir işletim sisteminde çalıştırmaya kadar soyutlama seviyesini yükseltir.
- **Loosely Coupled, distributed, elastic, liberated micro-services (Gevşek bağlantılı, dağıtılmış, esnek, özgürleştirilmiş mikro servisler):** Uygulamalar daha küçük, bağımsız parçalara ayrılır ve dinamik olarak dağıtılabilir ve yönetilebilir.
- **Resource Isolation (Kaynak İzolasyonu):** Öngörülebilir uygulama performansı
- **Resource Utilization (Kaynak Kullanımı):** Yüksek verimlilik ve yoğunluk.

4. Kubernetes Komponentleri



Şekil 4.1.

4.1. Temel Bileşenler

Bir Kubernetes kümesi (cluster), Bir Control Plane (Master-Node) ve bir veya daha fazla worker node'dan oluşur.

4.1.1. Control Plane (Kontrol Düzlemi- Master Node) Component

Küme hakkında genel kararların alındığı komponentleri barındırır.

4.1.1.1. kube-apiserver

Kubernetes'in merkezi API sunucusudur. Kullanıcılar ve diğer tüm bileşenler sadece kube-apiserver üzerinden iletişim kurarlar.

4.1.1.2. etcd

Tüm API sunucu verileri için tutarlı ve yüksek kullanılabilirliğe sahip anahtar-değer deposu.

4.1.1.3. scheduler

Kubernetes kümesinde oluşturulan ve henüz node ataması yapılmayan pod'ların kısıtlara göre hangi node üzerinde çalışacağını belirleyen kontrol düzlemi bileşenidir.

4.1.1.4. kube-controller-manager

Kubernetes kümesinin gerçek durumu ile istenilen durumların sürekli olarak uyumlu tutmak için çalışan kubernetes komponentidir.

4.1.1.5. cloud-controller-manager

Altta yatan bulut sağlayıcı/sağlayıcıları ile entegre olur.

4.1.2. Node (İşçi Düğüm- Worker Node) Component

Pod'ların çalıştığı makinedir.

4.1.2.1. kubelet

Pod içerisinde tanımlanan konteynerlerin başlatılmasını, çalışır durumda olmasını ve sağlık durumlarını kontrol eder.

4.1.2.2. kube-proxy

Servislerin uygulanması için node'larda ağ kurallarını korur.

4.1.3. Addons (Eklentiler)

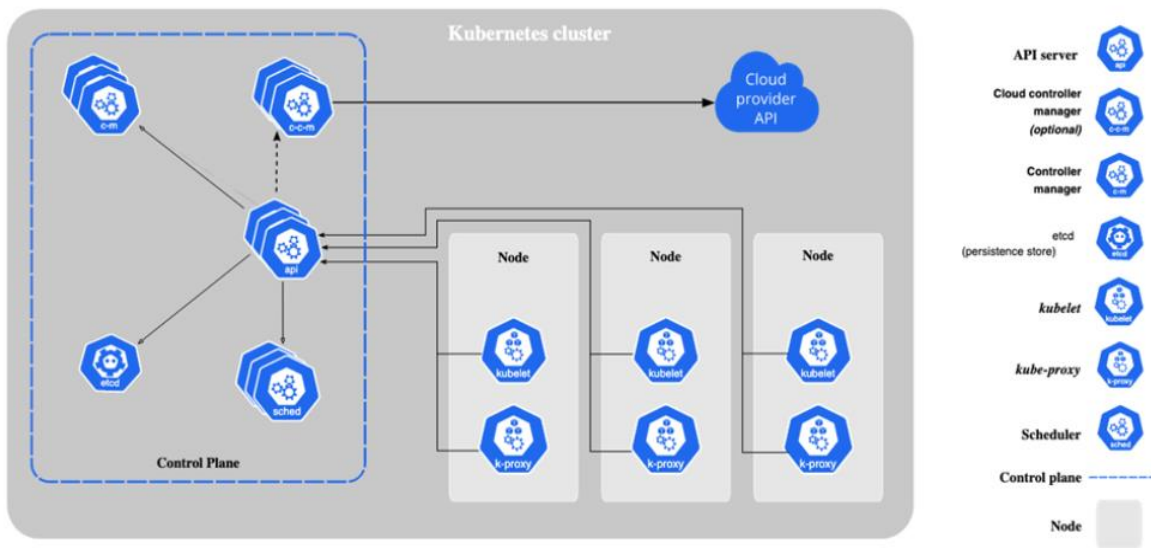
Eklentiler Kubernetes'in işlevselliğini genişletir. Birkaç önemli örnek şunlardır:

- **DNS:** Küme genelinde DNS çözümlemesi için.
- **WebUI:** Web arayüzü üzerinden küme yönetimi için.
- **Container Resource Monitoring:** Konteyner ölçümlerini toplamak ve saklamak için.

5. Kubernetes Küme Mimarisi

Bir Kubernetes kümesi, master node ve konteynerleştirilmiş uygulamaları çalıştıran worker node(lardan) adı verilen makinelerden oluşur. Her kümenin pod'ları çalıştırmak için en az bir worker node'a ihtiyacı vardır.

Worker node, uygulama iş yükünün bileşenleri olan Pod'ları barındırır. Master node, kümedeki worker node ve pod'ları yönetir. Üretim ortamlarında, master node genellikle birden fazla bilgisayarda çalışır ve bir küme genellikle birden fazla düğüm çalıştırarak hata toleransı ve yüksek kullanılabilirlik sağlar.



Şekil 5.1.

5.1. Control Plane (Kontrol Düzlemi- Master Node) Component

Kontrol düzlemi bileşenleri, küme hakkında küresel kararlar alır ve küme olaylarını algılar ve bunlara yanıt verir.

5.1.1. kube-apiserver

Kubernetes API sunucusunun ana uygulaması kube-apiserver'dır. Kube-apiserver yatay ölçeklenecek şekilde tasarlanmıştır; yani daha fazla örnek dağıtarak ölçeklenir. Birkaç kube-apiserver örneği çalıştırabilir ve bu örnekler arasında trafiği dengeleyebilirsiniz.

5.1.2. etcd

Kubernetes'in tüm küme verileri için depolama alanı olarak kullanılan, tutarı ve yüksek kullanılabilirliğe sahip anahtar-değer deposu.

5.1.3. kube-scheduler

Yeni oluşturulan ve henüz bir düğüme atanmamış Pod'ları izleyen ve bunların çalışacağı düğümü seçen kontrol düzlemi bileşenidir.

Planlama kararlarında dikkate alınan faktörler şunlardır: bireysel ve toplu kaynak gereksinimleri, donanım/yazılım/politika kısıtlamaları, yakınlık ve karşıt yakınlık özellikleri, veri yerelliği, iş yükler arası etkileşim ve son teslim tarihleri.

5.1.4. kube-controller-manager

Kontrol düzlemi bileşeni olup, kontrol süreçlerini çalıştırır.

Mantıksal olarak, her kontrolcü ayrı bir süreçtir, ancak karmaşıklığı azaltmak için hepsi tek bir ikili dosyaya derlenir ve tek bir süreçte çalıştırılır. Birçok farklı kontrolcü türü vardır. Bunlardan bazıları şunlardır:

- **Node Controller:** Düğümlerin devre dışı kaldığını fark etmekten ve yanıt vermekten sorumludur.
- **Job Controller:** Tek seferlik görevleri temsil eden "Job" nesnelerini izler ve çalıştırılmasını sağlar.
- **EndpointSlice Controller:** Servis ve podları eşler.
- **Replication Controller:** Pod sayısının Deployment/Statefulset konfigürasyon dosyasında belirtildiği gibi kopyalarının oluşturulup oluşturulmadığını kontrol eder
- **Service Account Controller:** Yeni namespace'ler için ServiceAccount nesnesi oluşturur.

5.1.5 cloud-controller-manager

Buluta özgü kontrol mantığını yerleştiren bir Kubernetes kontrol düzlemi bileşeni. cloud-controller-manager, kümenizi bulut sağlayıcınızın API'sine bağlamanıza ve bu bulut platformuyla etkileşim kuran bileşenleri, yalnızca kümenizle etkileşim kuran bileşenlerden ayırmanıza olanak tanır. cloud-controller-manager, yalnızca bulut sağlayıcınıza özgü denetleyicileri çalıştırır. Kubernetes'i kendi tesislerinizde veya kendi bilgisayarınızdaki bir öğrenme ortamında çalıştırıyorsanız, kümenin bir bulut denetleyici yöneticisi yoktur.

kube-controller-manager da olduğu gibi, cloud controller manager da mantıksal olarak bağımsız birkaç kontrol döngüsünü tek bir işlem olarak çalıştırdığınız tek bir ikili dosyada birleştirir. Performansı artırmak veya arızalara karşı toleransı artırmak için yatay olarak ölçeklendirebilirsiniz (birden fazla kopya çalıştırabilirsiniz).

Aşağıdaki denetleyicilerin bulut sağlayıcı bağımlılıkları olabilir:

- **Node Controller:** Bir node'un yanıt vermeyi bırakmasının ardından bulutta silinip silinmediğini belirlemek için bulut sağlayıcısını kontrol etmek için kullanılır.
- **Route Controller:** Temel bulut altyapısında rotaları ayarlamak için kullanılır.
- **Service Controller:** Bulut sağlayıcı yük dengeleyicilerini oluşturmak, güncellemek ve silmek için kullanılır.

5.2. Node (İşçi Düğüm- Worker Node)

Düğüm bileşenleri her düğümde çalışır, çalışan pod'ları sürdürür ve Kubernetes çalışma ortamını sağlar.

5.2.1. kubelet

Kümedeki her düğümde çalışır. Konteynerlerin bir pod içinde çalıştığından emin olur. Kubelet, Kubernetes tarafından oluşturulmayan kapsayıcıları yönetmez.

5.2.2. kube-proxy

kube-proxy, düğümlerde ağ kurallarını korur. Bu ağ kuralları, kümenizin içindeki veya dışındaki ağ oturumlarından Pod'larınıza ağ iletişimi sağlar. Servisler için paket iletimini kendi başına uygulayan ve kube-proxy'ye eşdeğer davranış sağlayan bir ağ eklentisi kullanıyorsanız, kümenizdeki düğümlerde kube-proxy çalıştırmanıza gerek yoktur.

5.2.3. container runtime

Kubernetes'in konteynerleri etkili bir şekilde çalıştırmasını sağlayan temel bir bileşendir. Kubernetes ortamında konteynerlerin yürütülmesini ve yaşam döngüsünü yönetmekten sorumludur.

Kubernetes, containerd, CRI-O ve Kubernetes CRI'nin (Konteyner Çalışma Zamanı Arayüzü) diğer tüm uygulamaları gibi konteyner çalışma zamanlarını destekler.

6. Konteynerler (Containers)

Uygulamanın çalışma zamanı bağımlılıklarıyla birlikte paketlenmesi için kullanılan teknoloji.

Bu sayfada konteynerler ve konteyner imajları ile bunların operasyon ve çözüm geliştirmedeki kullanımları ele alınacaktır.

“Konteyner” kelimesi çok anlam yüklü bir terimdir. Bu kelimeyi kullandığınızda, hedef kitlenizin aynı tanımı kullanıp kullanmadığını kontrol edin.

Çalıştırdığınız her konteyner tekrarlanabilir, bağımlılıkların dahil edilmesinden kaynaklanan standardizasyon, nerede çalıştırırsanız çalıştırın aynı davranışı elde etmenizi sağlar.

Konteynerler, uygulamaları altta yatan ana bilgisayar altyapısından ayırır. Bu, farklı bulut veya işletim sistemi ortamlarında dağıtımı kolaylaştırır.

Bir Kubernetes kümesindeki her düğüm, o düğüme atanan Pod'ları oluşturan konteynerleri çalıştırır. Bir Pod'daki konteynerler aynı düğümdede çalışacak şekilde birlikte konumlandırılır ve birlikte planlanır.

6.1. Konteyner İmajları (Container Images)

Bir konteyner imajı, bir uygulamayı çalıştırmak için gereken her şeyi içeren, çalışmaya hazır bir yazılım paketidir: kod ve gerektirdiği çalışma zamanı, uygulama ve sistem kütüphaneleri ve gerekli ayarlar için varsayılan değerler.

Konteynerler durumsuz ve değiştirilemez olacak şekilde tasarlanmıştır: Zaten çalışan bir konteynerin kodunu değiştirmemelisiniz. Konteynerleştirilmiş bir uygulamanız varsa ve değişiklik yapmak istiyorsanız, doğru işlem, değişikliği içeren yeni bir imaj oluşturmak ve ardından güncellenmiş imajdan başlamak için konteyneri yeniden oluşturmaktır.

6.2. Konteyner Çalışma Zamanı (Container Runtime)

Kubernetes'in konteynerleri etkili bir şekilde çalıştırmasını sağlayan temel bir bileşendir. Kubernetes ortamında konteynerlerin yürütülmesini ve yaşam döngüsünü yönetmekten sorumludur.

Kubernetes, containerd, CRI-O ve Kubernetes CRI'nin (Konteyner Çalışma Zamanı Arayüzü) diğer tüm uygulamaları gibi konteyner çalışma zamanlarını destekler.

Genellikle, kümenizin bir Pod için varsayılan konteyner çalışma zamanını seçmesine izin verebilirsiniz. Kümenizde birden fazla konteyner çalışma zamanı kullanmanız gerekiyorsa, Kubernetes'in bu konteynerleri belirli bir konteyner çalışma zamanı kullanarak çalıştırmasını sağlamak için bir Pod için RuntimeClass belirtebilirsiniz. Ayrıca, farklı Pod'ları aynı konteyner çalışma zamanı ile ancak farklı ayarlarla çalıştırmak için RuntimeClass kullanabilirsiniz.

7. Kubernetes Komponentleri

Kubernetes'te dağıtılabılır en küçük işlem nesnesi olan Pod'ları ve bunları çalıştırmanıza yardımcı olan daha üst düzey soyutlamaları anlayın.

Bir iş yükü, Kubernetes üzerinde çalışan bir uygulamadır. İş yükünüz tek bir bileşen veya birlikte çalışan birkaç bileşen olsun, Kubernetes'te onu bir dizi Pod içinde çalıştırırsınız. Kubernetes'te bir Pod, kümenizde çalışan bir veya daha fazla container kümesini temsil eder.

Kubernetes Pod'larının tanımlanmış bir yaşam döngüsü vardır. Örneğin, bir Pod kümenizde çalışmaya başladıktan sonra, o Pod'un çalıştığı düğümde kritik bir hata oluşması, o düğümdeki tüm Pod'ların başarısız olması anlamına gelir. Kubernetes bu seviyedeki hatayı son olarak ele alır: düğüm daha sonra sağlıklı hale gelse bile, kurtarmak için yeni bir Pod oluşturmanız gerekir.

Ancak, hayatı önemli ölçüde kolaylaştırmak için, her Pod'u doğrudan yönetmenize gerek yoktur. Bunun yerine, sizin adınıza bir dizi Pod'u yöneten iş yükü kaynaklarını kullanabilirsiniz. Bu kaynaklar, belirttiğiniz duruma uygun olarak doğru sayıda ve doğru türde Pod'un çalıştığından emin olan denetleyicileri yapılandırır.

Kubernetes, çeşitli yerleşik iş yükü kaynakları sağlar:

- **Deployment ve ReplicaSet:** Deployment, kümenizde durumsuz bir uygulama iş yükünü yönetmek için uygundur; burada Deployment'taki herhangi bir Pod gerekirse yenileriyle değiştirilebilir.

- **StatefulSet:** Bir şekilde durumu izleyen bir veya daha fazla ilgili Pod çalıştırmanıza olanak tanır. Örneğin, iş yükünüz verileri kalıcı olarak kaydediyorsa, her Pod'u bir PersistentVolume ile eşleştiren bir StatefulSet çalıştırabilirsiniz. Bu StatefulSet için Pod'larda çalışan kodunuz, genel dayanıklılığı artırmak için aynı StatefulSet'teki diğer Pod'lara veri kopyalayabilir.
- **DaemonSet:** düğümlere özgü tesisler sağlayan Pod'ları tanımlar. DaemonSet'te belirtilen özelliklere uyan bir düğümü kümenize her eklediğinizde, kontrol düzlemi bu DaemonSet için yeni düğümüne bir Pod planlar. Bir DaemonSet'teki her Pod, klasik bir Unix/POSIX sunucusundaki sistem daemon'una benzer bir iş gerçekleştirir. Bir DaemonSet, kümenizin çalışması için temel olabilir; örneğin, küme ağını çalıştırmak için bir eklenti olabilir, düğümü yönetmenize yardımcı olabilir veya çalıştırdığınız konteyner platformunu geliştiren isteğe bağlı davranışlar sağlayabilir.
- **Job ve CronJob:** Tamamlanana kadar çalışan ve ardından duran görevleri tanımlamanın farklı yollarını sunar. Bir Job kullanarak, yalnızca bir kez tamamlanana kadar çalışan bir görevi tanımlayabilirsiniz. Bir CronJob kullanarak, aynı Job'u bir programa göre birden fazla kez çalıştırabilirsiniz.

7.1. Pods

Podlar, Kubernetes'te oluşturabileceğiniz ve yönetebileceğiniz en küçük dağıtılabilir bilgi işlem birimleridir.

Bir Pod, paylaşılan depolama ve ağ kaynaklarına sahip bir veya daha fazla konteynerden oluşan bir gruptur ve konteynerlerin nasıl çalıştırılacağına dair bir spesifikasyona sahiptir. Bir Pod'un içeriği her zaman aynı yerde bulunur ve aynı zamanlamaya göre planlanır ve paylaşılan bir bağlamda çalışır. Bir Pod, uygulamaya özgü bir "mantıksal ana bilgisayar"ı modeller: nispeten sıkı bir şekilde birbirine bağlı bir veya daha fazla uygulama konteyneri içerir. Bulut dışı bağlamlarda, aynı fiziksel veya sanal makinede yürütülen uygulamalar, aynı mantıksal ana bilgisayarda yürütülen bulut uygulamalarına benzer.

Uygulama konteynerlerinin yanı sıra, bir Pod, Pod başlatma sırasında init container da içerebilir. Ayrıca, bir Pod'da hata ayıklama için ephemeral container da ekleyebilirsiniz.

7.1.1. Pod Nedir?

Pod'ların çalışabilmesi için kümedeki her düğüme bir container runtime kurmanız gerekir.

Bir Pod'un paylaşılan bağlamı, bir dizi Linux ad alanı, cgrup ve potansiyel olarak diğer izolasyon unsurlarından oluşur. Bir konteyneri izole eden şeylerle aynıdır. Bir Pod'un bağlamı içinde, bireysel uygulamalara daha fazla alt izolasyon uygulanabilir.

Bir Pod, paylaşılan ad alanlarına ve paylaşılan dosya sistemi birimlerine sahip bir konteyner kümesine benzer.

Kubernetes kümesindeki Pod'lar iki ana şekilde kullanılır:

- **Tek bir konteyner çalıştıran Pod'lar:** Pod başına bir konteyner" modeli, en yaygın Kubernetes kullanım durumudur; bu durumda, bir Pod'u tek bir konteynerin etrafındaki bir sarmalayıcı olarak düşünebilirsiniz; Kubernetes, konteynerleri doğrudan yönetmek yerine Pod'ları yönetir.
- **Birlikte çalışması gereken birden fazla konteyner çalıştıran pod'lar:** Bir Pod, birbirine sıkıca bağlı ve kaynakları paylaşması gereken, aynı yerde bulunan birden fazla konteynerden oluşan bir uygulamayı kapsayabilir. Bu aynı yerde bulunan konteynerler tek bir bütünleşik birim oluşturur.

Birden fazla aynı yerde bulunan ve birlikte yönetilen konteyneri tek bir Pod'da gruplandırmak nispeten gelişmiş bir kullanım durumudur. Bu modeli yalnızca konteynerlerinizin birbirine sıkıca bağlı olduğu belirli durumlarda kullanmalısınız.

7.1.2. Pod'ları Yönetmek İçin İş Yüğü Kaynakları

Genellikle, tekil Pod'lar da dahil olmak üzere Pod'ları doğrudan oluşturmanıza gerek yoktur. Bunun yerine, Deployment veya Job gibi iş yükü kaynaklarını kullanarak oluşturun. Pod'larınızın durumu izlemesi gerekiyorsa, StatefulSet kaynağını göz önünde bulundurun. Her Pod, belirli bir uygulamanın tek bir örneğini çalıştırmak için tasarlanmıştır. Uygulamanızı yatay olarak ölçeklendirmek (daha fazla örnek çalıştırarak daha fazla genel kaynak sağlamak) istiyorsanız, her örnek için bir tane olmak üzere birden fazla Pod kullanmalısınız. Kubernetes'te bu genellikle çoğaltma olarak adlandırılır.

Çoğaltılan Pod'lar genellikle bir iş yükü kaynağı ve denetleyicisi tarafından bir grup olarak oluşturulur ve yönetilir.

7.1.3. Pod Yaşam Döngüsü (Pod Lifecycle)

- **Pending:** Yeni oluşturulan iş yükünün/yüklerinin worker node'a atanmadan önce aldığı statü (Pod bu statü de kalıyorsa kube-scheduler uygun bir worker node bulamadığı anlamına gelir. Bu durumun çeşitli sebepleri olabilir).
- **Creating:** Worker-node'a atanan pod'un oluşturulma aşamasının başladığı anlamına gelir.
- **ImagePullBackOff:** Konteyner imajının ilgili repodan çekilememesi durumu (yanlış tanımlanma veya ilgili repoya yetki durumları).
- **Running:** Pod çalışmaya başladığını belirten statü.
- **CrashLoopBackOff:** Pod'un sürekli kapanması durumunda müdahale için set edilen statü.

7.1.4. Çoklu Konteyner Pod (Multi Container Pod)

Bir podun içerisinde birden fazla konteyner çalıştırılması durumudur. Genel olarak production ortamlarında sıkı sıkıya bağımlı olmadığı sürece bütün konteyner'lar tek bir pod içerisinde çalıştırılması uygundur: Tek pod içerisinde çoklu konteyner yaklaşımını kullanarak geliştirdiğiniz uygulamayı deploy ederseniz, uygulamaya yatay ölçeklendirme yapamazsınız. Bu yüzden önerilmez!

7.1.5. Init Konteyner

Init konteyner, pod içerisinde tanımladığınız konteyner başlatılmadan önce başlayan bir konteyner türüdür. Init container ile tanımlanan tüm işlemler yapılır, yapıldıktan sonra kapanır. Init konteyner kapanınca pod içerisinde tanımlamış olduğunuz konteyner çalışmaya başlar. Init konteyner tanımlanan işlemlerini tamamlamadan konteyner'iniz başlatılmaz.

NOT:

- **Label (Etiketler):** Etiketler, nesneleri (Pod, Service, ReplicaSet vb.) organize etmek, gruplamak ve seçmek için kullanılan anahtar-değer çiftleridir. Aynı zamanda etiketler sayesinde kubernetes nesnelerinin birbirleriyle olan ilişkilerini tanımlarken de kullanırız.
- **Annotation (Açıklama):** Annotation'lar, nesnelere sorgulama amacı gütmeyen, büyük ve serbest formatta meta veriler eklemek için kullanılır. Annotation'ların en güçlü ve yaygın kullanım amacı, k8s ekosistemine sonradan dahil olan üçüncü parti araçlar ile nesneleriniz arasında bir köprü kurmaktır.

Aralarındaki Fark: Üçüncü parti bir araç, hangi pod ile konuşacağını bulmak için Label'ları kullanabilir, ancak Pod'a özel bir yapılandırma, kural veya konfigürasyon uygulamak istediğinde bunu Annotation üzerinden okur.

7.2. Deployment

Bir Deployment, genellikle durum korumayan (stateless) uygulama iş yüklerini çalıştırmak için bir dizi Pod'u yönetir.

Bir Deployment da istediğiniz durumu tanımlarsınız ve Deployment Controller, gerçek durumu kontrollü bir hızda istenen duruma değiştirir. Yeni ReplicaSet'ler oluşturmak veya mevcut Deployment'ları kaldırmak ve güncellenen kaynak konfigürasyonlarını tüm Deployment'lara özümsetmek için Deployment tanımlayabilirsiniz.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Şekil 7.1. Deployment Örneği

7.3. Services

Kubernetes'te, kümenizde çalışan bir uygulamayı, iş yükü birden fazla arka uçta bölünmüş olsa bile, tek bir dışa dönük uç nokta üzerinden erişilebilir hale getirme yöntemidir.

Kubernetes de Servis, kümenizde bir veya daha fazla pod olarak çalışan bir ağ uygulamasını erişilebilir hale getirme yöntemidir.

Kubernetes de uygulamanızı çalıştırmak için Deployment kullanıyorsanız, bu Deployment pod'ları dinamik olarak oluşturulabilir ve yok edilebilir. Bir andan diğerine, bu podlardan kaçının çalışır durumda ve sağlıklı olduğunu bilemezsiniz; hatta bu sağlıklı pod'ların adlarının ne olduğunu bilmiyor olabilirsiniz. Kubernetes Pod'ları, kümenizin istenen durumuna uyacak şekilde oluşturulur ve yok edilir. Pod'lar geçici kaynaklardır (tek bir pod'un güvenilir ve kalıcı olmasını beklememelisiniz).

Her pod kendi IP adresine sahiptir (Kubernetes, ağ eklentilerinin bunu sağlaması bekler). Kümenizdeki belirli bir Deployment için, bir anda çalışan Pod'lar bir an sonra aynı uygulamayı çalıştıran pod'lar kümesinden farklı olabilir.

Bu bir soruna yol açar: Eğer bazı pod'lar (bunlara backend diyelim) kümenizdeki diğer pod'lara (bunlara frontend diyelim) işlevsellik sağlıyorsa, frontend hangi IP adreslerine bağlanacaklarını nasıl bulup takip ederler ki, frontend backend iş yükünün bir kısmını kullanabilsin? İşte burda Servisler devreye giriyor.

7.3.1. Kubernetes'te Servisler

Kubernetes'in bir parçası olan Servis API'si, Pod gruplarını bir ağ üzerinden kullanıma sunmanıza yardımcı olan bir soyutlamadır. Her servis nesnesi, mantıksal bir uç nokta kümesi (genellikle bu uç noktalar Pod'lardır) ve bu Pod'lara nasıl erişileceğine dair bir politika tanımlar.

Örneğin, 3 kopyayla çalışan durumsuz bir görüntü işleme backend uygulaması düşünün. Bu kopyalar değiştirilebilirdir, frontend uygulamaları hangi backend podlarını kullandıklarıyla ilgilenmezler. Backend kümesini oluşturma gerçek pod'lar değişebilirken, frontend istemcilerinin bunun farkında olmalarına veya backend kümesini takip etmelerine gerek yoktur. Servis soyutlaması bu ayrıştırmayı sağlar.

Bir servis tarafından hedeflenen Pod kümesi genellikle tanımladığınız bir selector tarafından belirlenir.

İş yükünüz HTTP ile konuşuyorsa, web trafiğinin bu iş yüküne nasıl ulaşacağını kontrol etmek için bir Ingress kullanmayı tercih edebilirsiniz. Ingress bir servis türü değildir, ancak kümeniz için giriş noktası görevi görür. Bir Ingress, yönlendirme kurallarınızı tek bir kaynakta birleştirmenize olanak tanır, böylece kümenizde ayrı ayrı çalışan iş yükünüzün birden fazla bileşenini tek bir dinleyicinin arkasında açığa çıkarabilirsiniz.

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Şekil 7.2. Örnek Servis Tanımı

7.3.2. Servis Türleri

- **ClusterIP:** Servisi küme içi bir IP adresinde sunar. Bu değeri seçmek, Servisin sadece küme içinden erişilebilir olmasını sağlar. Bu, bir Servis için açıkça bir tür belirtmediğinizde kullanılan varsayılan değerdir. Servisi bir Ingress kullanarak genel internete açabilirsiniz.
- **NodePort:** Servisi her düğümün IP adresinde statik bir portta (NodePort) sunar. Düğüm portunu kullanılabilir hale getirmek için Kubernetes, ClusterIP türünde bir servis istemiş gibi bir Cluster IP adresi ayarlar.
- **LoadBalancer:** Servisi harici bir yük dengeleyici kullanarak dışarıya sunar. Kubernetes doğrudan yük dengeleme bileşeni sunmaz. Bunu kendiniz sağlamanız veya Kubernetes kümenizi bir bulut sağlayıcısıyla entegre etmeniz gerekir.

7.3.3. Headless Service (Başsız Servisler)

Bazen yük dengelemesine ve tek bir Servis IP adresine ihtiyacınız olmayabilir. Bu durumda, küme IP adresi açıkça “None” (spec.ClusterIP) belirterek headless service’ler oluşturabilirsiniz.

Kubernetes’in kendi uygulamasına bağımlı kalmadan, diğer servis keşif mekanizmalarıyla arayüz oluşturmak için bir headless service kullanabilirsiniz.

Headless Service’ler için bir ClusterIP tahsis edilmez, kube-proxy bu servisleri işlemez ve platform tarafından bunlar için herhangi bir yük dengeleme veya yönlendirme yapılmaz.

Bir headless service, bir istemcinin hangi pod’a isterse ona doğrudan bağlanmasına olanak tanır. Headless olan servisler, sanal IP adresleri ve proxy’ler kullanarak rotalar ve paket yönlendirmeleri yapılandırmazlar, bunun yerine, kümenin DNS servisi aracılığıyla sunulan dahili DNS kayıtları vasıtasıyla, bağımsız pod’ların doğrudan uç nokta IP adreslerini bildirirler. Bir headless service tanımlamak için .spec.type.clusterIP alanını “None” ayarlayın.

None string değeri özel bir durumdur ve spec.type.clusterIP alanını boş bırakmayla aynı şey değildir.

DNS’in otomatik olarak nasıl yapılandırılacağı, Servis’in tanımlanmış selector’lere sahip olup olmadığına bağlıdır:

- **Selector’lü:** Selector tanımlayan headless servisler için kontrol düzleminde bulunan uç nokta denetleyicisi (endpoints controller), Kubernetes API’sinde EndpointSlice nesneleri oluşturur ve DNS yapılandırmasını, Doğrudan Servis’in arkasındaki Pod’ları işaret eden A veya AAAA kayıtlarını (IPv4 veya IPv6 adresleri) döndürecek şekilde değiştirir.
- **Selector’süz:** Selector tanımlanmayan headless Servisler için kontrol düzlemi EndpointSlice nesneleri oluşturmaz. Ancak DNS sistemi aşağıdakilerden birini arar ve yapılandırır:
 - ExternalName türündeki servisler için DNS CNAME kayıtları

- ExternalName dışındaki tüm servis türleri için, Servis'in hazır durumdaki uç noktalarının tüm IP adreslerine yönelik DNS A/AAAA kayıtları

Selector'süz bir headless servis tanımladığınızda, port değeri ile targetPort değerinin eşleşmesi zorunludur.

7.4. Secret

Kubernetes Secret, parolalar, OAuth token ve ssh anahtarları gibi hassas bilgileri depolamanıza ve yönetmenize olanak tanır. Gizli bilgileri bir Secret içinde saklamak onu bir Pod tanımına veya bir container imajına koymaktan daha güvenli ve esnektir.

Secret'lar, onları kullanan Pod'lardan bağımsız olarak oluşturulabildiğinden, Pod'ların oluşturulması, görüntülenmesi ve düzenlenmesi iş akışı sırasında Secret'ın açığa çıkma riski daha azdır. Kubernetes ve kümenizde çalışan uygulamalar, hassas verileri kalıcı depolamaya yazmaktan kaçınmak gibi Secret'lar ile ilgili ek önlemler de alabilir.

Secret'lar, ConfigMaps'e benzer ancak özellikle gizli verileri tutmak için tasarlanmıştır.

7.5. ConfigMap

ConfigMap, gizli olmayan verileri anahtar-değer eşlenikleri olarak depolamak için kullanılan bir API nesnesidir. Podlar, ConfigMap'i environment variable, komut satırı argümanları veya bir volume olarak bağlanan yapılandırma dosyaları olarak kullanabilir.

7.6. DaemonSet

DaemonSet, tüm (veya bazı) node'ların bir Pod'un bir kopyasını çalıştırmasını sağlar. Cluster'a yeni node eklendikçe, onlara Podlar'da eklenir. Cluster'dan node kaldırıldığında bu pod'lar da kaldırılır. Bir DaemonSet'in silinmesi, oluşturulduğu Pod'ları da temizleyecektir.

7.7. Volume

Kubernetes Volume'leri, bir Pod içindeki konteynerlerin dosya sistemi aracılığıyla verilere erişmesine ve bunları paylaşmasına olanak tanır. Farklı amaçlar için kullanabileceğiniz farklı volume türleri vardır:

- ConfigMap veya Secret'a dayalı olarak bir konfigürasyon dosyası doldurma.
- Bir Pod için geçici bir depolama alanı sağlama.
- Aynı Pod içindeki iki farklı konteyner arasında dosya sistemini paylaşma
- İki farklı Pod arasında dosya sistemini paylaşma (bu Pod'lar farklı düğümlerde çalışsa bile).
- Pod yeniden başlatılsa veya değiştirilse bile verilerin kullanılabilir kalması için verileri kalıcı olarak depolama.

Veri paylaşımı, bir konteyner içindeki farklı yerel işlemler arasında, farklı konteynerler arasında veya Pod'lar arasında olabilir.

7.7.1. Volume'lerin Önemi

- **Veri Kalıcılığı (Data Persistence):** Konteyner üzerindeki dosyalar geçicidir ve bu durum, konteynerlerde çalışan karmaşık uygulamalar için bazı sorunlar yaratır. Bir sorun, konteyner çöktüğünde veya durdurulduğunda ortaya çıkar; konteyner durumu kaydedilmez (stateless), bu nedenle konteynerin ömrü boyunca oluşturulan veya değiştirilen tüm dosyalar kaybolur. Bir çökmeden sonra kubelet konteyneri temiz bir durumla yeniden başlatır.
- **Paylaşılan Depolama (Shared Storage):** Başka bir sorun, bir Pod'da birden fazla konteyner çalışırken ve dosyaları paylaşmaları gerektiğinde ortaya çıkar. Tüm konteynerler arasında paylaşılan bir dosya sistemi kurmak ve erişmek zor olabilir.

Kubernetes volume soyutlaması, bu sorunların her ikisini de çözmenize yardımcı olabilir.

7.7.2. Volume'ler Nasıl Çalışır?

Kubernetes birçok volume türünü destekler. Bir Pod, aynı anda herhangi bir sayıda volume türünü kullanabilir. Geçici volume (ephemeral volume) türlerinin ömrü belirli bir Pod ile bağlantılıdır, ancak kalıcı volume'ler (persistent volume) herhangi bir Pod'un ömrünün ötesinde varlığını sürdürür. Bir Pod'un varlığı sona erdiğinde, Kubernetes geçici volume'leri yok eder. Ancak kubernetes kalıcı volume'leri yok etmez. Belirli bir Pod'daki herhangi bir volume türü için veriler, konteyner yeniden başlatmaları arasında korunur.

Bir Pod başlatıldığında, konteynerdeki bir işlem, konteyner görüntüsünün ilk içeriğinden ve konteynerin içine bağlanmış volume'lerden (eğer volume tanımlaması yapılmışsa)

oluşan bir dosya sistemi görür. Bu işlem, başlangıçta konteyner görüntüsünün içeriğiyle eşleşen bir kök dosya sistemi görür. İzin verilirse, bu dosya sistemi hiyerarşisi içindeki herhangi bir yazma işlemi, işlemin daha sonraki bir dosya sistemi erişimi gerçekleştirdiğinde gördüğü şeyi etkiler. Volume'ler, konteyner içindeki dosya sistemi içindeki belirtilen yollara bağlanır. Bir Pod içinde tanımlanan her konteyner için, konteynerin kullandığı her volume'un nereye bağlanacağını bağımsız olarak belirtmeniz gerekir.

7.8. Persistent Volumes

Depolama yönetimi (Storage management), bilgi işlem örneklerini yönetmekten farklı bir sorundur. PersistentVolume alt sistemi, kullanıcılar ve yöneticiler için depolamanın nasıl sağlandığına dair detayları, bu depolamanın nasıl tüketildiğinden soyutlayan bir API sunar. Bunu gerçekleştirmek için iki yeni API kaynağı sunuyoruz: PersistentVolume ve PersistentVolumeClaim.

Bir PersistentVolume, küme içerisinde bir yönetici tarafından önceden ayrılmış (provisioned) veya Storage Classes (Depolama Sınıfları) kullanılarak dinamik olarak ayrılmış bir depolama alanıdır. Tıpkı bir Node'un küme içi kaynak olması gibi, PV de küme içinde bir kaynaktır. PV'lerde Volume'ler gibi bir Volume eklentisidir. Ancak PV'yi kullanan herhangi bir pod'dan bağımsız bir yaşam döngüsüne sahiptirler. Bu API nesnesi, NFS, iSCSI veya bulut sağlayıcısına özgü bir depolama sistemi fark etmeksizin, depolamanın uygulanışına dair teknik detayları kendi içinde barındırır.

PersistentVolumeClaim, bir kullanıcı tarafından yapılan depolama talebidir. Pod'lara oldukça benzer. Pod'lar Node kaynaklarını tüketirken PVC'ler de PV kaynaklarını tüketir. Pod'lar belirli seviyelerde kaynak (CPU ve Bellek) talep edebilir, Claim'ler (Talepler) ise belirli boyut ve erişim modları (access modes) talep edebilir (Örneğin, ReadWriteOnce, ReadOnlyMany, ReadWriteMany veya ReadWriteOncePod olarak bağlanabilirler.)

PersistentVolumeClaims, kullanıcıların soyut depolama kaynaklarını tüketmelerine olanak tanıyor olsa da kullanıcıların farklı problemler için genellikle performans gibi değişen özelliklere sahip PersistentVolumes yapılarına ihtiyacı olur. Küme yöneticilerinin, kullanıcıları bu birimlerin arka planda nasıl uygulandığına dair detaylara maruz bırakmadan; boyut ve erişim modlarından çok daha fazla kritere göre farklılaşan

çeşitli PersistentVolumes seçenekleri sunabilmesi gerekir. Bu tür ihtiyaçlar için StorageClass kaynağı mevcuttur.

7.8.1. Bir Volume ve Claim'in Yaşam Döngüsü

PV'ler küme içindeki kaynaklardır. PVC'ler ise bu kaynaklara yapılan taleplerdir ve aynı zamanda kaynağa erişim sağlayan birer “talep fişi (claim check)” görevi görürler. PV'ler ve PVC'ler arasındaki etkileşim şu yaşam döngüsünü takip eder:

Provisioning (Kaynak Sağlama)

PV'lerin sağlanmasının (provisioned) iki yolu vardır: statik veya dinamik olarak.

Statik

Bir küme yöneticisi (cluster administrator) belirli sayıda PV oluşturur. Bu PV'ler, küme kullanıcılarının kullanımına hazır olan gerçek depolama alanının detaylarını taşırlar. Kubernetes API'sinde yer alırlar ve tüketilmeye (kullanılmaya) hazırdırlar.

Dinamik

Yöneticinin oluşturduğu statik PV'lerden hiçbirinin bir kullanıcının PersistentVolumeClaim talebiyle eşleşmediği durumlarda küme, doğrudan o PVC için özel olarak dinamik bir volume sağlamayı deneyebilir. Bu kaynak sağlama işlemi StorageClasses yapısına dayanır: PVC'nin bir Storage Class talep etmesi ve dinamik sağlamanın gerçekleştirilmesi için yöneticinin bu sınıfı önceden oluşturup yapılandırmış olması gerekir. Storage Class'ı "" (boş string) olarak talep eden Claim'ler kendileri için dinamik kaynak sağlamayı etkili bir şekilde devre dışı bırakmış olurlar.

Depolama sınıfına dayalı dinamik depolama sağlamayı etkinleştirmek için, küme yöneticisinin API sunucusunda DefaultStorageClass kabul denetleyicisini (admission controller) etkinleştirmesi gerekir. Bu, örneğin, API sunucu bileşenini --enable-admission-plugins bayrağına ait virgülle ayrılmış ve sıralı değerler listesinde DefaultStorageClass parametresinin yer alması sağlanarak yapılabilir.

Binding (Bağlama)

Bir kullanıcı, belirli miktarda depolama alanı ve belirli erişim modları talep eden bir PersistentVolumeClaim oluşturur. Kontrol düzlemindeki bir kontrol döngüsü yeni PVC'leri izler, (eğer mümkünse) eşleşen bir PV bulur ve bunları birbirine bağlar. Eğer yeni bir PVC için dinamik olarak bir PV sağlanmışsa, döngü o PV'yi her zaman ilgili PVC'ye

bağlayacaktır. Aksi takdirde (statik sağlamada), kullanıcı her zaman en az talep ettiği kadarını alır, ancak atanan birimin boyutu talep edilenden daha fazla olabilir.

Bir kez bağlandıktan sonra, PersistentVolumeClaim bağları, nasıl bağlandıklarına bakılmaksızın özeldir (exclusive). Bir PVC ile PV arasındaki bağlama işlemi; PersistentVolume ile PersistentVolumeClaim arasında çift yönlü bir bağ oluşturan bir ClaimRef kullanılarak yapılan bire bir (one-to-one) eşleşmedir.

Eşleşen bir birim bulunmadığı sürece Claim'ler süresiz olarak bağlanmamış (unbound) olarak kalacaktır. Claim'ler, eşleşen birimler kullanılabilir hale geldikçe bağlanır. Örneğin, çok sayıda 50Gi boyutunda PV ile donatılmış bir küme, 100Gi talep eden bir PVC ile eşleşmeyecektir. Bu PVC, kümeye 100Gi boyutunda bir PV eklendiğinde bağlanabilir.

Using (Kullanım)

Pod'lar, Claim'leri birer Volume olarak kullanırlar. Küme, bağlı olan volume'ü bulmak için claim'i inceler ve bu volume'u ilgili pod için bağlar. Birden fazla erişim modunu destekleyen volume'ler için kullanıcı, talebini bir pod içinde volume olarak kullanırken hangi modun geçerli olmasını istediğini belirtir.

Bir kullanıcı bir claim'e sahip olduğunda ve bu claim bağlandığında, bağlı olan PV kullanıcının ihtiyacı olduğu sürece ona ait olur. Kullanıcılar, Pod'un volumes bloğuna bir persistentVolumeClaim bölümü ekleyerek Pod'ları schedule eder. Ve talep ettikleri PV'lere erişirler.

Kullanımdaki Depolama Nesnesi Koruması

"Kullanımdaki depolama nesnesi koruması" özelliğinin amacı; bir Pod tarafından aktif olarak kullanılan PersistentVolumeClaim (PVC) yapılarının ve bu PVC'lere bağlı olan PV yapılarının sistemden silinmesini engellemektir, çünkü bu durum veri kaybına yol açabilir.

NOT: Bir PVC'yi kullanan bir pod nesnesi mevcut olduğu sürece, o PVC bir pod tarafından aktif kullanımda (active use) kabul edilir.

Eğer bir kullanıcı, bir Pod tarafından aktif kullanımda olan bir PVC'yi silerse, PVC sistemden hemen kaldırılmaz. PVC'nin kaldırılması, söz konusu PVC artık hiçbir Pod

tarafından aktif olarak kullanılmayana kadar ertelenir. Benzer şekilde, eğer bir yönetici bir PVC'ye bağlı olan bir PV'yi silerse, PV sistemden hemen kaldırılmaz. PV'nin kaldırılması, PV artık bir PVC'ye bağlı olmayana kadar ertelenir.

Reclaiming (Geri Alma)

Bir kullanıcı volume ile işi bittiğinde, kaynağın geri kazanılmasına izin PVC nesnelerini API'den silebilir. Bir PersistentVolume için belirlenen geri kazanım politikası (reclaim policy), küme yapısına, claim serbest bırakıldıktan sonra volume'e ne yapılması gerektiğini söyler. Şu anda volume'ler Retained (tutulan), Recycled (Geri dönüştürülen) veya Deleted (Silinen) politikalarına sahip olabilirler. Recycle kullanımdan kaldırıldığı için değinilmemiştir.

Retain (Tutmak)

Retain geri kazanım politikası, kaynağın manuel olarak geri kazanılmasına olanak tanır. PersistentVolumeClaim silindiğinde, PersistentVolume var olmaya devam eder ve volume "serbest bırakılmış (released)" olarak kabul edilir. Ancak, önceki talep sahibinin verileri hala volume üzerinde bulunduğundan, bu volume henüz başka bir claim için kullanılabilir durumda değildir. Bir yönetici şu adımları izleyerek volume'ü manuel olarak geri kazanabilir:

1. PV nesnesini siler. PV nesnesi silindikten sonra bile dış altyapıdaki (external infrastructure) ilişkili depolama varlığı var olmaya devam eder.
2. İlişkili depolama varlığı üzerindeki verileri uygun şekilde manuel olarak temizler.
3. İlişkili depolama varlığını manuel olarak siler.

Eğer aynı depolama varlığını yeniden kullanmak istiyorsanız, aynı depolama varlığı tanımına sahip yeni bir PV oluşturmanız gerekir.

Delete (Silmek)

Delete geri kazanım politikalarını destekleyen volume eklentileri için silme işlemi hem PV nesnesini Kubernetes'ten kaldırır hem de dış altyapıdaki ilişkili depolama varlığını siler. Dinamik olarak sağlanan volume'ler, varsayılan Delete olan kendi StorageClass yapılarının geri kazanım politikasını miras alır. Yönetici, StorageClass yapısını kullanıcıların beklentilerine göre yapılandırmalıdır. Aksi takdirde, PV oluşturulduktan sonra düzenlenmeli veya yamalanmalıdır.

PersistentVolume Rezerv Etme

Kontrol düzlemi, PVC yapılarını kümedeki eşleşen PV yapılarına otomatik olarak bağlayabilir. Ancak, bir PVC'nin belirli bir PV'ye bağlanmasını istiyorsanız, bunları önceden birbirine bağlamanız gerekir.

Bir PVC içinde belirli bir PV belirterek, o özel PV ile PVC arasında bir bağ deklarasyonu yapmış olursunuz. Eğer ilgili PersistentVolume mevcutsa ve calimRef alanı üzerinden başka bir PersistentVolumeClaim için rezerve edilmemişse, bu PersistentVolume ve PersistentVolumeClaim birbirine bağlanır.

Bu bağlama işlemi, düğüm yakınlığı (node affinity) dahil olmak üzere bazı volume eşleştirme kriterlerinden bağımsız olarak gerçekleştirilir. Yine de kontrol düzlemi, Storage class'ın, erişim modlarının ve talep edilen depolama boyutunun geçerli olup olmadığını kontrol etmeye devam eder.

Bu yöntem tek başına PersistentVolume üzerinde herhangi bir bağlama ayrıcalığını garanti etmez. Eğer belirttiğiniz PV'yi başka PersistentVolumeClaim yapılarının da kullanma ihtimali varsa, öncelikle o depolama birimini rezerve etmeniz gerekir. Diğer PVC'lerin bu birime bağlanmasını engellemek için, PV'nin calimRef alanında ilgili PersistentVolumeClaim'i belirtmeniz gerekir.

NOT:

Bir küme içinde PersistentVolume kullanımı için, volume türüyle ilgili yardımcı programlara ihtiyaç duyulabilir. NFS dosya sistemlerinin abğlanmasını desteklemek için /sbin/mount.fs yardımcı programına ihtiyaç duyulmaktadır.

Erişim Modları

ReadWriteOnce(RWO)

Volume, tek bir node tarafından okunabilir-yazılabilir olarak bağlanabilir. ReadWriteOnce erişim modu, Pod'lar aynı düğüm üzerinde çalıştığı sürece, birden fazla Pod'un o birime erişmesine (okumasına-yazmasına) yine de izin verebilir.

ReadOnlyMany (ROX)

Volume, birçok node tarafından salt okunur (readonly) olarak bağlanabilir.

ReadWriteMany(RWX)

Volume, birçok node tarafından okunabilir-yazılabilir olarak bağlanabilir.

ReadWriteOncePod(RWOP)

Volume, yalnızca tek bir Pod tarafından okunabilir-yazılabilir olarak bağlanabilir. Tüm küme genelinde yalnızca tek bir pod'un o PVC'yi okuyabilmesini veya ona yazabilmesini garanti etmek istiyorsanız ReadWriteOncePod erişim modunu kullanın.

Not:

Bir PV aynı anda birden fazla erişim modunu destekliyor olsa bile, bir PVC tarafından bind edildiği an yalnızca tek bir erişim moduyla kullanılabilir.

Not:

Kubernetes, PVC ve PV'yi eşleştirmek için volume erişim modlarını kullanır. Bazı durumlarda, volume erişim modları PV'nin nereye mount edileceğini de kısıtlar. Birim erişim modları, depolama birimi mount edildikten sonra yazma korunmasını zorunlu kılmaz. Erişim modları ReadWriteOnce, ReadOnlyMany veya ReadWriteMany olarak belirtilmiş olsa bile, volume üzerinde herhangi bir kısıtlama getirmezler. Örneğin, bir PV ReadOnlyMany olarak oluşturulmuş olsa bile, salt okunur olacağının garantisi yoktur. Ancak Erişim modları ReadWriteOncePod olarak belirtilmişse, volume kısıtlanır ve yalnızca tek bir pod'a monte edilir

Selector (Seçici)

Claim'ler, kullanılabilir durumdaki volume kümesini daha da daraltmak için bir etiket seçici (label selector) belirtebilir. Yalnızca etiketleri seçici ile eşleşen volume'ler ilgili claim'e bağlanabilir. Selector iki alandan oluşabilir:

- **MatchLabels:** Volume, tam olarak bu değerde bir label'a sahip olmalıdır.
- **MatchExpressions:** Anahtar, değerler listesi ve bu anahtar ile değerleri ilişkilendiren bir operatör belirtilerek oluşturulan gereksinimler listesidir. Geçerli operatörler arasında In, NotIn, Exists ve DoesNotExist yer alır.

Class (Sınıf)

Bir Claim, storageClassName özniteliğini kullanarak belirli bir StorageClass adı belirtebilir ve o sınıfı talep edebilir. Yalnızca talep edilen sınıfa ait olan, yani PVC ile aynı storageClassName değerine sahip PV'ler ilgili PVC'ye bağlanabilir.

PVC'lerin mutlaka bir sınıf talep etmesi zorunlu değildir. storageClassName değeri "" (boş string) olarak ayarlanan bir PVC, her zaman "hiçbir sınıfı olmayan" bir PV talep ediyor olarak yorumlanır. Bu nedenle yalnızca sınıfı olmayan (herhangi bir anotasyonu bulunmayan veya anotasyonu "" olarak ayarlanmış) PV'lere bağlanabilir.

7.9. Storage Classes (Depolama Sınıfları)

Bir StorageClass, yöneticilerin sundukları depolama sınıflarını tanımlamalarına olanak tanır. Farklı sınıflar; hizmet kalitesi seviyelerine, yedekleme politikalarına veya küme yöneticileri tarafından belirlenen keyfi politikalara eşlenebilir. Kubernetes'in kendisi, bu sınıfların neyi temsil ettiği konusunda herhangi bir görüş belirtmez veya kısıtlama getirmez.

Kubernetes'teki depolama sınıfı kavramı, diğer bazı depolama sistemi tasarımlarındaki “profiller” yapısına benzer.

7.9.1. StorageClass Objeleri

Her StorageClass, ilgili sınıfa ait bir PV yapısının, bir PVC talebini karşılamak üzere dinamik olarak sağlanması gerektiğinde kullanılan provisioner (sağlayıcı), parameters (parametreler) ve reclaimPolicy (geri kazanım politikası) alanlarını içerir.

Bir StorageClass nesnesinin adı önemlidir ve kullanıcıların belirli bir sınıfı talep etme yöntemidir. Yöneticiler, StorageClass nesnelerini ilk kez oluştururken sınıfın adını ve diğer parametrelerini belirlerler.

Bir yönetici olarak, belirli bir sınıf talep etmeyen tüm PVC'lere uygulanacak varsayılan bir StorageClass (default StorageClass) belirtebilirsiniz.

7.9.2. Varsayılan StorageClass (Default StorageClass)

Bir StorageClass yapısını kümeniz için varsayılan (default) olarak işaretleyebilirsiniz. Bir PVC bir storageClassName belirtmediğinde, varsayılan StorageClass kullanılır.

Kümenizde birden fazla StorageClass üzerinde storageclass.kubernetes.io/is-default-class anotasyonunu true olarak ayarlarsanız ve ardından storageClassName alanı belirtilmemiş bir PVC oluşturursanız, Kubernetes en son oluşturulan default StorageClass yapısını kullanır.

NOT:

Kümenizde varsayılan olarak işaretlenmiş yalnızca tek bir StorageClass bulundurmaya özen göstermelisiniz. Kubernetes'in birden fazla varsayılan StorageClass bulunmasına izin vermesinin nedeni, kesintisiz ve sorunsuz geçişlere olanak tanımadır.

Yeni bir PVC için storageClassName belirtmeden de bir PersistentVolumeClaim oluşturabilirsiniz; bunu kümenizde hiçbir varsayılan StorageClass mevcut olmadığında bile yapabilirsiniz. Bu durumda, yeni PVC tanımladığınız şekilde oluşturulur ve varsayılan bir sınıf kullanılabilir hale gelene kadar bu PVC'nin storageClassName alanı ayarlanmamış olarak kalır.

Herhangi bir varsayılan StorageClass bulunmayan bir kümeye sahip olabilirsiniz. Eğer hiçbir depolama sınıfını varsayılan olarak işaretlemesiniz (ve örneğin bir bulut sağlayıcısı sizin adınıza bir tane atamadıysa), Kubernetes bu ihtiyaç duyan PersistentVolumeClaims yapıları için bu varsayılan atama işlemlerini uygulayamaz.

Varsayılan bir StorageClass kullanılabilir hale geldiğinde, kontrol düzlemi storageClassName alanı bulunmayan mevcut PVC'leri tespit eder. storageClassName alanı boş bir değere sahip olan veya bu anahtarı hiç barındırmayan PVC'ler için kontrol düzlemi, bu PVC'leri varsayılan StorageClass ile eşleştirecek şekilde günceller. Eğer storageClassName değeri "" (boş string) olan mevcut bir PVC'niz varsa ve siz varsayılan bir StorageClass yapılandırırsanız, bu PVC güncellenmeyecektir.

Varsayılan bir StorageClass mevcutken bile storageClassName değeri "" olarak ayarlanmış PV'lere bağlanmaya devam etmek istiyorsanız, ilgili PVC'nin storageClassName alanını "" olarak ayarlamanız gerekir.

7.9.3. Provisioner (Sağlayıcı)

Talep ettiğin diski bulut servis sağlayıcılar veya nfs (buluttan veya kendi sunucundan) sistemlerde gidip otomatik oluşturan mekanizmadır.

7.9.4. Reclaim Policy (Geri Alma Politikası)

PVC silindiğinde diskin içindeki verinin durumunu belirler iki seçeneği vardır:

- Delete (Varsayılan): PVC silinirse, arka plandaki gerçek disk de verilerle birlikte tamamen silinir.
- Retain: PVC silinse bile disk ve içindeki veriler korunur, yönetici manuel olarak temizleyene kadar sistemde bekler.

7.9.5. Volume Expansion (Birim Geniřletme)

PV yapıları genişletilebilir olacak şekilde yapılandırılabilir. Bu ilgili pvc nesnesini düzenleyip daha büyük miktarda yeni bir depolama alanı talep ederek volume’u boyutlandırmanıza olanak tanır.

Arka plandaki “allowVolumeExpansion: true” olarak ayarlandığında, desteklenen volume türleri volume genişletmeyi uygulayabilir.

7.9.6. Mount Options (Bağlama Seçenekleri)

Bir StorageClass tarafından dinamik olarak oluşturulan PV yapıları, sınıfın “mountOptions” alanında belirtilen bağlama seçeneklerine sahip olur.

Eğer volume eklentisi bağlama seçeneklerini desteklemiyorsa ancak buna rağmen bağlama seçenekleri belirtilmişse, kaynak sağlama (provisioning) işlemi başarısız olur. Bağlama seçenekleri ne sınıf ne de PV üzerinde doğrulanmaz. Eğer bir bağlama seçeneği geçersizse, PV bağlama işlemi başarısız olur.

7.9.7. Volume Binding Mode (Birim Bağlama Modu)

“volumeBindingMode” alanı, volume bağlama ve dinamik kaynak sağlama işlemlerinin ne zaman gerçekleşmesi gerektiğini kontrol eder. Ayarlanmadığında, varsayılan olarak Immediate (hemen) modu kullanılır.

- **Immediate Modu:** PVC oluşturulur oluşturulmaz volume bağlama ve dinamik kaynak sağlama işlemlerinin gerçekleşeceğini belirtir.
- **WaitForFirstConsumer Modu:** Küme yöneticisi, bir PV’un bağlanmasını ve sağlanmasını, bu PVC’yi kullanan bir pod oluşturulana kadar erteyecek olan WaitForFirstConsumer modunu belirterek bu sorunu çözebilir.

NOT:

Eğer WaitForFirstConsumer modunu kullanmayı seçerseniz, düğüm yakınlığı (node affinity) belirtmek için Pod spesifikasyonunda (Pod spec) nodeName parametresini kullanmayın. Bu durumda nodeName kullanılırsa, zamanlayıcı devre dışı bırakılacak ve PVC Pending durumunda kalacaktır. Bunun yerine, kubernetes.io/hostname için nodeSelector kullanabilirsiniz.

7.9.8. Parametrs (Parametreler)

StorageClass, sınıfa ait volume'lerin özelliklerini tanımlayan parametrelere sahiptir. Kullanılan sağlayıcıya (provisioner) bağlı olarak farklı parametreler kabul edilebilir ([Daha fazlası için Bkz.](#)) Bir parametre belirtilmediğinde (atlandığında), o parametre için belirlenmiş bir varsayılan değer kullanılır.

Bir StorageClass için en fazla 512 parametre tanımlanabilir. Anahtarlar ve değerler dahil olmak üzere parametreler nesnesinin toplam uzunluğu 256KiB sınırını aşamaz.

NFS

NFS depolamasını yapılandırmak için Kubernetes ile yerleşik gelen sürücüyü veya Kubernetes için geliştirilmiş NFS CSI (önerilen) kullanabilirsiniz.

- **server:** NFS sunucusunun ana bilgisayar adı (hostname) veya IP adresidir.
- **path:** NFS sunucusu tarafından paylaşılan dizin yoludur.
- **readOnly:** Depolama biriminin salt okunur olarak bağlanıp bağlanmayacağını belirten bir flag'dir (varsayılan değer false yani hem okunabilir hem yazılabilir.)

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: example-nfs
provisioner: example.com/external-nfs
parameters:
  server: nfs-server.example.com
  path: /share
  readOnly: "false"
```

Şekil 7.1.

Özetle:

Statik bağlama PV,PVC şemasını oluşturursunuz nfs driver kurmanıza gerek kalmaz.

Dinamik Bağlama PVC, StorageClass şemasını oluşturursunuz NFS CSI vb. driver kurarsınız arka planda PV otomatik olarak oluşur.

7.10. StatefulSets

Bir StatefulSet, bir grup pod'u çalıştırır ve bu Pod'ların her biri için sabit bir kimlik sürdürür. Bu yapı, kalıcı depolamaya (persistent storage) veya kararlı, benzersiz bir ağ kimliğine ihtiyaç duyan uygulamaları yönetmek için kullanışlıdır.

StatefulSet, stateful (durumsal) uygulamaları yönetmek için kullanılan iş yükü API nesnesidir. Bir grup pod'un dağıtımını ve ölçeklenmesini yönetir, bu Pod'ların sıralılığı ve benzersizliği hakkında garantiler sağlar.

Bir Deployment gibi, bir StatefulSet de aynı konteyner spesifikasyonuna dayalı pod'ları yönetir. Ancak bir Deployment'tan farklı olarak StatefulSets, Podlarının her biri için sabit bir kimlik korur. Bu Pod'lar aynı spesifikasyondan oluşturulur, ancak birbirlerinin yerine geçemezler: Her biri, yeniden zamanlama işlemleri boyunca koruduğu sabit bir kimliğe sahiptir.

İş yükünüz için kalıcılık sağlamak amacıyla depolama birimlerini kullanmak istiyorsanız, çözümün bir parçası olarak StatefulSet kullanabilirsiniz. Bir StatefulSet içindeki bireysel Pod'lar arızlanmaya açık olsalar da kalıcı Pod kimlikleri, arızlanan Pod'ların yerine geçen yeni Pod'larla mevcut volume'leri eşleştirmeyi kolaylaştırır.

7.10.1. StatefulSet Kullanımı

StatefulSet'ler aşağıdakilerden bir veya daha fazlasına ihtiyaç duyan uygulamalar için değerlidir:

- Kararlı, benzersiz ağ tanımlıyıcıları
- Kararlı, kalıcı depolama
- Sıralı, sorunsuz dağıtım ve ölçeklendirme
- Sıralı, otomatik kademeli güncellemeler.

Yukarıdaki maddelerde geçen "kararlı" ifadesi Pod'un yeniden zamanlanması (silinip tekrar ayağa kalkması) süreçlerinde kalıcılığın korunması ile eş anlamlıdır. Eğer bir uygulama herhangi bir kararlı bir kimliğe veya sıralı dağıtım, sıralı silme veya sıralı ölçeklendirme işlemlerine ihtiyaç duymuyorsa, uygulamanızı stateless (durumsuz) replikalar sağlayan bir iş yükü nesnesi kullanarak dağıtmalısınız. Durumsuz ihtiyaçlar için Deployment daha uygun olabilir.

7.10.2. Kısıtlamalar

- Belirli bir Pod içinde kullanılacak depolama alanı, ya talep edilen StorageClass'a dayalı bir PersistentVolume Provisioner (Kalıcı birim sağlayıcı) tarafından dinamik olarak sağlanmalı ya da bir yönetici tarafından önceden manuel oluşturulmuş olmalıdır.
- Bir StatefulSet'in silinmesi ve/veya ölçek küçültülmesi, StatefulSet ile ilişkili olan volume'leri silmez. Bu kural, veri güvenliğini sağlamak amacıyla konulmuştur, çünkü verinin korunması, ilişkili tüm StatefulSet kaynaklarının otomatik olarak temizlenmesinden genellikle çok daha değerlidir.
- StatefulSet'ler, Pod'ların ağ kimliğini yönetmek için şu anda bir HeadlessService'e (başsız servis) ihtiyaç duyar. Bu Servis'i oluşturma sorumluluğu siz aittir.
- StatefulSet'ler, StatefulSet nesnesinin kendisi silindiğine Pod'ların sonlandırılma sırasına dair herhangi bir garanti vermez. Pod'ların sıralı ve sorunsuz bir şekilde sonlandırılmasını sağlamak için, StatefulSet nesnesini silmeden önce replika sayısını 0'a indirerek ölçek küçültme mümkündür.

7.11. Ingress

Ingress kavramı, Kubernetes API'si aracılığıyla tanımladığınız kurallara dayanarak trafiği farklı arka uçlara (backends) eşlemenizi sağlar.

7.11.1. Terimler

Netlik sağlamak adına, bu kılavuz aşağıdaki terimleri tanımlar:

- **Node (Düğüm):** Kubernetes tarafından yönetilen, konteynerleştirilmiş uygulamaları çalıştıran bir dizi düğümdür. Bu örnek için en yaygın Kubernetes dağıtımlarında, kümedeki düğümler genel internetin bir parçası değildir.
- **Edge Router (Kenar Yönlendirici):** Kümeniz için güvenlik duvarı (firewall) politikasını uygulayan bir yönlendiricidir. Bu, bir bulut sağlayıcısı tarafından yönetilen bir ağ geçidi veya fiziksel bir donanım parçası olabilir.
- **Cluster Network (Küme Ağı):** Kubernetes ağ modeline göre bir küme içindeki iletişimi kolaylaştıran, mantıksal veya fiziksel bağlantılar kümesidir.

- **Service (Servis):** Etiket seçici (label selectors) kullanarak bir dizi Pod'u tanımlayan bir Kubernetes servsidir. Aksi belirtilmedikçe, Servislerin yalnızca küme ağı içinde yönlendirilebilir ClusterIP'lere sahip olduğu varsayılır.

7.11.2. Ingress Nedir?

Ingress, küme dışından küme içindeki servislere giden HTTP ve HTTPS rotalarını dış dünyaya açar (exposes). Trafik yönlendirmesi, Ingress kaynağı üzerinde tanımlanan kurallarla kontrol edilir.

İşte bir Ingress'in tüm trafiği tek bir servise gönderdiği basit bir örnek:



Bir Ingress, servislere dışarıdan erişilebilir URL'ler sağlamak, trafiği yük dengelemek, SSL/TLS sonlandırmak ve isim tabanlı sanal barındırma sunmak üzere yapılandırılabilir. Bir Ingress nesnesinin gereksinimlerini yerine getirmekten genellikle bir yük dengeleyici ile çalışan Ingress Controller sorumludur.

7.11.3. Ön Koşullar

Bir Ingress tanımının çalışabilmesi için kümenizde mutlaka bir Ingress Controller kurulu olmalıdır. Sadece bir Ingress nesnesi oluşturmanın sistem üzerinde hiçbir etkisi yoktur. Çok sayıda Ingress Controller (Örneğin, Traefik, NGINX Ingress, HAProxy vb.) arasından seçim yapabilirsiniz. İdeal olarak, tüm Ingress Controller'lar referans spesifikasyona uyması gerekir. Ancak gerçekte, çeşitli Ingress Controller'lar biraz farklı şekilde çalışır.

7.11.4. Ingress Kaynağı

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```

Bir Ingress nesnesi, apiVersion, kind, metadata ve spec alanlarına ihtiyaç duyar.

Ingress Controller'lar, davranışlarını yapılandırmak için sıklıkla anotasyonları kullanır. Hangi anotasyonların beklendiğini ve/veya desteklendiğini öğrenmek için seçtiğiniz ingress controller'un dokümantasyonunu inceleyin.

Spec alanı, bir yük dengeleyici veya proxy suncusunu yapılandırmak için gereken tüm bilgileri içerir. En önemlisi, gelen tüm isteklere eşleştirecek bir kurallar listesi (rules list) barındırır. Ingress kaynağı yalnızca HTTP(S) trafiğini yönlendirmeye yönelik kuralları destekler.

Eğer ingressClassName alanı boş bırakılırsa (belirtilmezse), tanımlanmış bir varsayılan Ingress sınıfı kullanılmalıdır. Bazı ingress controller'lar, varsayılan bir IngressClass tanımı olmasa bile çalışabilir. Her ne kadar herhangi bir IngressClass olmadan çalışabilen bir ingress controller kullanıyor olsanız bile, Kubernetes projesi yine de varsayılan bir IngressClass tanımlamanızı önermektedir.

7.11.4.1. Ingress Kuralları

- **İsteğe bağlı bir host (ana bilgisayar adı):** Eğer bir host belirtilmezse (bu örnekte olduğu gibi), kural belirtilen IP adresi üzerinden gelen tüm HTTP trafiğine

uygulanır. Eğer bir Host sağlanırsa (örneğin foo.bar.com), kurallar yalnızca o host'a uygulanır.

- **Bir yollar (paths) listesi:** (Örneğin /testpath). Her yolun, bir service.name ile bir service.port.name veya service.port.number ile tanımlanmış ilişkili bir arka ucu vardır. Yük dengeleyicinin trafiği ilgili servise yönlendirmesi için, gelen isteğin içeriğinin hem host hem de path tanımıyla eşleşmesi gerekir.
- **Backend (Arka uç):** Servis dökümanında açıklandığı gibi bir servis ve port adının birleşimidir veya bir CRD (Custom Resource Definition) aracılığıyla tanımlanmış özel bir path ile eşleşen HTTP(S) istekleri, listelenen backend'e gönderilir.

Bir defaultBackend, genellikle spec alanındaki hiçbir yolla eşleşmeyen istekleri karşılamak üzere Ingress Controller içinde yapılandırılır.

7.11.4.1. DefaultBackend

Gelen isteklerin hiçbir Ingress kuralıyla (host/path) eşleşmediği durumlarda isteği karşılayan yedek (fallback) servistir. Ingress kaynağında spec.defaultBackend alanı ile tanımlanır ve genellikle özel bir 404 sayfası döndürmek için kullanılır.

7.11.4.3. Resource Backend

İnternette gelen trafiği alışlagelmiş şekilde bir Kubernetes Service nesnesine değil, doğrudan küme içindeki **başka bir kaynağa** (en yaygın örneğiyle statik dosyaların durduğu bir Object Storage/MinIO nesnesine) bağlama yöntemidir.

7.11.4.4. Path Types

Ingress içindeki her yolun (path) buna karşılık gelen bir yol tipine (path type) sahip olması zorunludur. Açıkça bir pathType içermeyen yollar doğrulama aşamasından geçemez ve hata verir. Desteklenen üç yol türü vardır.

- **ImplementationSpecific (Uygulamaya Özel):** Bu yol türünde eşleştirme mantığı tamamen kullanılan IngressClass'a (yani kurduğun Nginx, Traefik vb. controller'a) bağlıdır. Kontroller uygulamaları bunu ayrı bir pathType olarak ele alabilir veya Prefix ya da Exact yol türleriyle tamamen aynı şekilde değerlendirebilir.
- **Exact (Birebir eşleşme):** URL yolunu büyük/küçük harf duyarlılığına (case sensitivity) dikkat ederek birerbir olarak eşleştirir.

- **Prefix (Önek Eşleşmesi):** URL yolunu / işaretiyle ayrılmış öenklere göre eşleştirir. Eşleştirme büyük küçük harf duyarlıdır ve yol ögesi bazında yapılır. Bir yol ögesi, yolun içindeki / ayracı ile bölünmüş etiketlerin her birini ifade eder. Eğer bir p yolu, gelen istek yolunun öge bazındaki her bir öneki ile eşleşiyors o istek p yolu için bir eşleşme sayılır.

- **Not:** Eğer tanımlanan yolun son ögesi, gelen istek yolundaki sın öğenin sadece bir alt metni ise bu bir eşleşme sayılmaz. (Örneğin: /foo/bar tanımı /foo/bar/baz isteği ile eşleşir, ancak /foo/barbaz isteğiyle eşleşmez.)

Exact ve Prefix Karşılaştırması için Bkz. “[PathTypes](#)”

7.11.5. TLS

Bir Ingress nesnesini, içinde TLS özel anahtarı ve sertifikası barındıran bir Secret belirterek güvenli hale getirebilirsiniz. Ingress kaynağı yalnızca tek bir TLS portunu destekler (443) ve TLS sonlandırmasının (TLS termination) giriş noktasında yapıldığını varsayar (yani Ingress’ten içerideki Servise ve Pod’lara giden trafik şifresiz olarak gider).

Eğer Ingress içindeki TLS yapılandırma bölümünde farklı host’lar belirtilmişse, bu host’lar SNI (Server Name Indication) TLS uzantısı aracılığıyla belirtilen ana bilgisayar adına göre aynı port (443) üzerinden çoğullanır -Tabii kullanılan Ingress Controller’un SNI desteklediği sürece. TLS Secret’ı, kullanılacak sertifika ve özel anahtarı içermek üzere tls.crt ve tls.key adında anahtarlara sahip olmalıdır.

Not:

Çeşitli ingress kontrolcülerinin desteklediği TLS özellikleri arasında farklılıklar/boşluklar vardır. Kendi ortamınızda TLS’in nasıl çalıştığını tam olarak anlamak için seçtiğiniz ingress kontrolcüsünün (kontrolcülerinin) dökümantasyonuna başvurmalısınız.

```
apiVersion: v1
kind: Secret
metadata:
  name: testsecret-tls
  namespace: default
data:
  tls.crt: base64 encoded cert
  tls.key: base64 encoded key
type: kubernetes.io/tls
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-example-ingress
spec:
  tls:
  - hosts:
    - https-example.foo.com
    secretName: testsecret-tls
  rules:
  - host: https-example.foo.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: service1
            port:
              number: 80
```

Liveness Probe

Kubernetes üzerinde çalışan iş yükleri running state de olmalarına rağmen beklenmedik şekillerde davranışlar gösterdiği durumlarda kubelet komponenti ilgili pod/podların durum/durumlarını running gördüğü için restart etmez. Bu problemin önüne geçebilmek için Liveness Probe kullanılır. 7.3.3. Liveness Probe

Readiness Probe

Kubelet, bir konteynerin ne zaman trafiği kabul etmeye hazır olduğunu bilmek için Readiness Probe'ları kullanır. Bir pod, tüm konteynerler hazır olduğunda hazır kabul edilir. Readiness probe sayesinde bir pod hazır olana kadar service arkasına eklenmez.

Resource Limits

Konteynerler aksini belirtmediğiniz sürece, çalışmış olduğu ortamdaki kaynakların tamamını tüketebilir. Bu durum verimsizdir. Bu durumun önüne geçebilmek için iş yükleri içerisinde resource limits ile iş konteynerlerin kullanacakları kaynakları sınırlandırabiliriz.

```
resources:
  requests:
    memory: "64M"
    cpu: "250m"
  limits:
    memory: "256M"
    cpu: "0.5"
```

Node affinity

Node affinity kavramsal olarak nodeSelector'e benzer: node'lara atanan etiketlere göre Podunuzun hangi node üstünde schedule edilmeye uygun olduğunu kısıtlamanıza olanak tanır.

Pod affinity

Pod affinity, Podunuzun hangi node üstünde oluşturulmaya uygun olduğunu, node'lardaki etiketlere göre değiş, halihazırda node'da çalışmakta olan pod'lardaki etiketlere göre sınırlamamıza olanak tanır.